

Laborator 6 - Proiectarea Algoritmilor

METODA DIVIDE ET IMPERA

1. Problema acoperirii cu piese în forma de "L" a unei table de șah $n \times n$,
 $n = 2^k$.
2. Se citește de la tastatură vectorul TATĂ asociat unui arbore (TATĂ[rădăcină]=0).
Scrieți un program care verifică dacă:
 - (a) arborele dat este un arbore binar;
 - (b) arborele dat este un arbore binar strict.
3. PROBLEMA CĂUTĂRII (implementarea algoritmilor prezentați în curs):
 - (a) Căutarea secvențială.
 - (b) Căutarea binară - varianta recursivă.
 - (c) Căutarea binară - varianta iterativă.
 - (d) Algoritmul de căutare $A(T)$.
4. PROBLEMA SORTĂRII (implementarea algoritmilor prezentați în curs):
 - (a) Sortarea prin numărarea frecvențelor componentelor.
 - (b) Sortarea prin inserție directă.
 - (c) Sortarea prin inserție binară.
 - (d) Interclasarea a doi vectori;
 - (e) Sortarea prin interclasare (2 algoritmi);
 - (f) Sortarea rapidă (2 algoritmi);

APLICATII REZOLVATE – METODA DIVIDE ET IMPERA

Test de autoevaluare:

1. Ce legătură presupune metoda *Divide et Impera* între tipul problemei și tipul subproblemelor?
2. Dați o descriere succintă a metodei *Divide et Impera*, bazată pe arbori.
3. De ce este aplicabilă metoda *Divide et Impera* în calculul paralel ?
4. Care este factorul cu care se reduce timpul de executare a unei probleme prin aplicarea metodei *Divide et Impera* ?

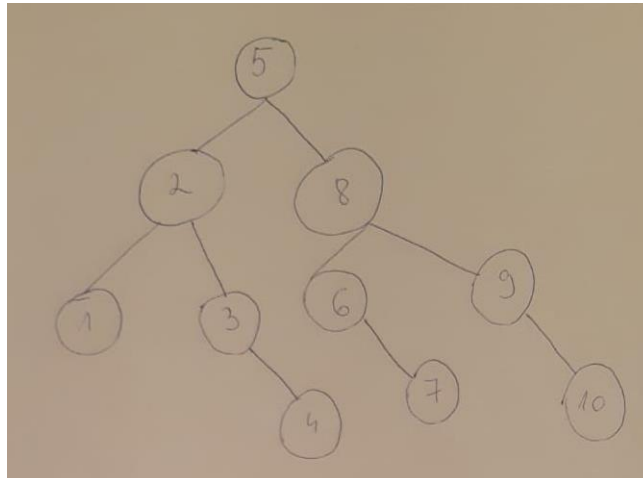
Raspunsul la testul de autoevaluare:

1. Metoda *Divide et Impera* presupune că tipul unei probleme coincide totdeauna cu tipul subproblemelor în care este descompusă (de care depinde).
2. Metoda *Divide et Impera* constă în:
 - construirea dinamică a unui arbore (prin împărțirea în subprobleme);
 - parcurgerea în postordine a arborelui (prin asamblarea rezultatelor parțiale).
3. Rezolvarea unei probleme poate fi făcută în modul următor:
 - sunt delegate procesoare pentru rezolvarea subproblemelor;
 - după ce se obțin rezultatele subproblemelor, procesorul care se ocupă de problemă asamblează aceste rezultate.
4. În general, dacă timpul necesar executării unui algoritm este $O(n^k)$, transformarea sa prin aplicarea metodei *Divide et Impera* conduce la un timp de ordinul $O(n^{k-1} \cdot \log n)$.

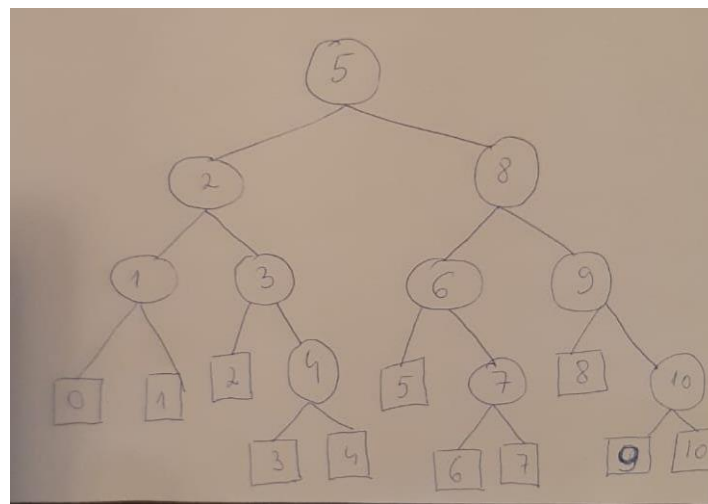
APLICATIA 1 - Problema cautarii -

1. Construiti arborele binar de cautare asociat algoritmului cautarii binare pentru $n = 10$.

Observatie: Elementul median este $m = \left\lfloor \frac{1+n}{2} \right\rfloor = \left\lfloor \frac{1+10}{2} \right\rfloor = \left\lfloor \frac{11}{2} \right\rfloor = [5,5] = 5$



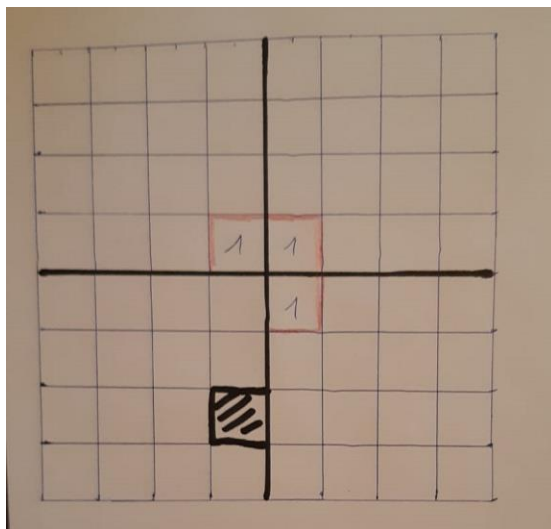
2. Construiti arborele binar extins de cautare asociat algoritmului cautarii binare pentru $n = 10$.



APLICATIA 2 - Problema de acoperire-

$n=8$, patratul unitar acoperit este pe linia 7, coloana 4

SOLUTIE: Plasam piesa 1 in centru astfel incat fiecare dintre cele 4 patrate de dimensiune 4 sa aiba cate un patrat unitar acoperit:



Continuam procedeul pentru fiecare dintre cele 4 patrate de dimensiune 4:

3	3	4	4	8	8	9	9
3	2	2	4	8	7	7	9
5	2	6	6	10	10	7	11
5	5	6	1	1	10	11	11
13	13	14	14	1	18	19	19
13	12	12	14	18	18	17	19
15	12	16		20	17	17	21
15	15	16	16	20	20	21	21

APLICATIA 3: Suma elementelor dintr-un vector.

```
#include <iostream>

using namespace std;

int Suma(int V[], int st, int dr)
{
    if(st == dr)
        return V[st]; // problemă elementară
    else
    {
        int m = (st + dr) / 2; // împărțim problema în subprobleme
        int s1 = Suma(V, st, m); // rezolvăm prima subproblemă
        int s2 = Suma(V, m + 1, dr); // rezolvăm a doua subproblemă
        return s1 + s2; // combinăm rezultatele
    }
}

int main()
{
    int V[101], n;
    cin>>n;

    int i;
    for (i = 1; i <= n; i++)
        cin>>V[i];
    cout << Suma(V,1,n);
    return 0;
}
```

APLICATIA 4: *Să se calculeze suma $S=1*2 + 2*3 + 3*4 + + n*(n+1)$.*

```
#include <iostream>

using namespace std;

int n;

void dei(int p,int u,int &z)
{
    int m,x1,x2;

    if(p==u)
        z=p*(p+1);
    else
    {
        m=(p+u)/2; //divizeaza
        dei(p,m,x1);
        dei(m+1,u,x2);
        z=x1+x2; //combina
    }
}

int main()
{
    int z=0;

    cin>>n;

    dei(1,n,z);

    cout<<z;

}
```